

Cars24 Mobile Engineering – The SDUI Assignment

****One assignment. One complex page. Your architecture.****

****Timebox: 72 hours from receiving this brief.**** Expected focused effort: ~8-10 hours – AI leverage is what makes that possible, and that's the point. How you use AI is half the assessment.

The problem

Every layout change to our pages costs us a full release cycle – build, review, staged rollout, and waiting for users to update – on Android and iOS separately. We want ****Server-Driven UI****: the server sends JSON, the client renders the page. Change the JSON → the page changes on every user's device, no app release.

Your job: ****design and build an SDUI system, prove it's fast, and prove it generalizes.****

The screen

Open the ****Cars24 app**** and pick a real screen as your reference – the ****home/landing page is a good choice****. Replicate it, rendered entirely by your SDUI system from JSON.

Whatever screen you pick must meet this complexity bar (the landing page clears all of these):

- At least ****5 visually distinct section types**** (e.g., search/header, banners, category chips, car card rails, value-prop strips, footer CTAs)
- At least one ****horizontal carousel/rail**** and one ****vertical list or grid****
- At least one ****interactive element driven by SDUI actions**** – a tab/chip selection that changes content, a tappable card with a navigation intent, etc. – not hardcoded behavior
- Real-feeling data (hardcode it in your JSON; nobody expects live APIs)

State in your README ****which screen you chose and why****. Picking a trivially simple screen is itself a judgment signal – choose something that shows your system off.

****Heads-up, and design for it:**** in the first round we will give you a ****different Cars24 screen**** and check exactly where your system needs changes – what renders with JSON edits only, and where you'd have to write client code. Build your schema for that

moment, not just for the screen you picked.

Mock the server: a local JSON file, embedded server, or hosted endpoint – your call. **The JSON schema is yours to design.** That schema is the heart of this assignment.

Platform choice

Build your SDUI renderer in **one** stack of your choice – Android/Kotlin, iOS/Swift, or React Native/TypeScript – and build it deeply. Implementing the same schema on a **second platform** is a significant bonus – it proves the schema is genuinely platform-neutral – but a deep single-stack build beats shallow multi-stack ones.

What your system must handle

- **Component registry** – server names a component type + props; client maps it to a native view
- **Actions** – taps, selections, navigation intents expressed in JSON (e.g., the tenure selector updating the EMI, the CTA opening the sheet)
- **Unknown component fallback** – server sends a type your client doesn't know: the page must degrade gracefully, never crash. Show us this working.
- **Versioning story** – how old app versions coexist with new server payloads (a section in your README is enough; implementation is bonus)

Part 2 – Prove it's fast

SDUI that's slow is worse than no SDUI. You must **also build a static (hardcoded) version of your chosen screen** on your primary stack, and benchmark both:

Metric	Definition (measure both versions, release build, same device)
TTR	Cold open → page fully rendered above the fold
TTI	Cold open → page scrollable and tappable
Full page time	Open → all sections rendered
SDUI breakdown	JSON fetch/parse time vs view-build time
Scroll perf	Dropped frames / jank while scrolling the full page

Report results in **PERF.md**: device used, methodology, numbers for static vs SDUI, and the **overhead %**. There is no pass/fail number – we're scoring measurement honesty and whether

you **optimized** after measuring (tell us what you tried, what worked, what didn't).

Part 3 – Prove it generalizes

The whole point of SDUI is the **next** screen. Submit `**`COVERAGE.md`**`:

- Your component registry and what UI patterns your schema can express (lists, grids, conditionals, actions, styling overrides...)
- An honest coverage claim: `**"Given a new Cars24 screen, X% renders with JSON-only changes; these patterns need new client code."**`
- `**In the first round, we will give you a Cars24 screen you didn't build.**` You'll write JSON for it in front of us. We measure: what % renders with zero code changes, what needs a new component, and how fast you (with your AI tools) add it.

Part 4 – AI workflow evidence (weighted like the code)

``AI_WORKFLOW.md`` in your repo:

- Tool stack and the `**context/rules files**` you wrote to brief your AI
- `**Three prompt→outcome stories**` – real prompts, what AI produced, `**what you rejected or rewrote and why**`
- `**One AI failure**` – where AI led you wrong (schema design, perf, platform API) and how you caught it
- Verification strategy for AI-generated code

Submission

- GitHub repo, meaningful commit history (we read how you worked)
- ``README.md`` – setup, architecture overview, schema design rationale, versioning story, trade-offs
- ``PERF.md``, ``COVERAGE.md``, ``AI_WORKFLOW.md``
- `**Screen recording (3–5 min):**` page rendering from JSON, the tenure selector + bottom sheet working, the unknown-component fallback, and one live edit – change the JSON, re-run, show the page change without touching client code

How we evaluate

Dimension	Weight	What great looks like
--- --- ---		
Architecture & solution quality	30%	Clean schema design, working interactions, graceful fallback, honest versioning story
AI collaboration	30%	Sharp briefs, rejected outputs, credible failure story, verification habits

Generalization (coverage)	20%	High JSON-only coverage on the surprise page; fast, calm extension when code is needed
Performance rigor	10%	Honest baseline methodology; evidence of measure→optimize loop
Ownership & judgment	10%	Explains every line in debrief; scoped smartly for the timebox

****What impresses us:**** a schema we'd actually adopt; overhead numbers reported honestly even when unflattering; fallback demo; JSON-edit live demo.

****What doesn't:**** a renderer hardcoded to this one page wearing a JSON costume; perf claims with no methodology; "the AI wrote that part."

FAQs

****Can I use an existing SDUI library?**** You may study any of them (Airbnb Epoxy/Lona posts, Lyft, Judo, etc.), but the schema and renderer must be your own — we're hiring you to design ours.

****Is 72 hours real?**** Yes — with real AI leverage. This assignment is sized so that an engineer who works AI-natively finishes comfortably and one who codes everything by hand cannot. Ruthless scoping is part of the test: cutting a section with a written trade-off note beats shipping all eight badly.

****One platform really OK?**** Yes. Depth first. Multi-platform is bonus, not baseline.